



数字逻辑与处理器基础

Fundamentals of Digital Logic and Processor



第二讲 数的表示与布尔代数

李学清

xueqingli@tsinghua.edu.cn

助教：曾令安*/唐文骏/陈仲豪

*本节内容负责助教

2024年9月21日

通知、答疑

- 后选课的同学
 - 自学、网络学堂**补交**第一次作业
- **微信实名**

唐文骏, 158 0118 6917, twj21@mails.tsinghua.edu.cn, 电子工程馆7-305

陈仲豪, 183 5975 7529, czh23@mails.tsinghua.edu.cn, 电子工程馆4-301

曾令安, 188 0135 7562, zla23@mails.tsinghua.edu.cn, 电子工程馆7-305

注意：7层西侧与弧形中厅走廊常锁，请通过其他层前往西侧，再走楼梯/电梯到达

习题课/答疑：

1. 周六晚7:00 – 9:00集中答疑、习题课
 - 答疑每周1次（第2周起），系馆9-206
 - 习题课时间地点待定，每主题一次，讲习题中问题，讲解后继续答疑
2. 网络学堂课程讨论区答疑
3. 助教/教师邮件答疑

课程回顾

– 为什么IC和CPU如此重要？有需求

- 用有限的电路结构解决各式各样的计算
- 设计约束和指标需求

– 如何应对这些需求

- 数字化、同步化、结构化
- 定制硬件和通用软件
- 使用EDA工具

– 为什么IC发展地如此迅猛

- 摩尔定律，架构创新

– 课程培养目标

- 获取知识，掌握技能，实践思想与方法论

布尔代数的定位

- The ‘big-bang’ starts with ‘0’ and ‘1’
 - In *complementary metal-oxide-semiconductor (CMOS)* technology

周	日期	暂定主要内容
1	9/9	绪论
2	9/21	布尔代数 (中秋节调休)
3	9/23	组合逻辑
4	9/30	时序逻辑
5	10/7	国庆节放假
6	10/14	时序逻辑
7	10/21	计算机指令系统
8	待定	期中考试 (TBD)
9	11/4	计算机指令系统
10	11/11	微处理器
11	11/18	微处理器
12	11/25	流水线
13	12/2	流水线
13	12/7	流水线/存储器 (调课)
15	12/16	存储器
16	12/23	存储器/IO与总线/总结

本节课目录

- 数的编码与二进制表示
- 算术与函数的布尔代数表示
- 布尔表达式的化简

编码和解码

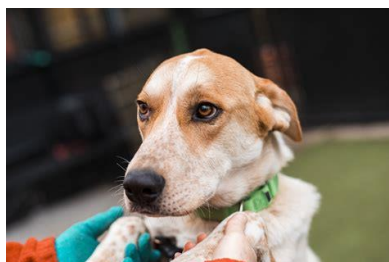
某种信息格式
e.g. 图像、视频、音频

编码(Encoding)



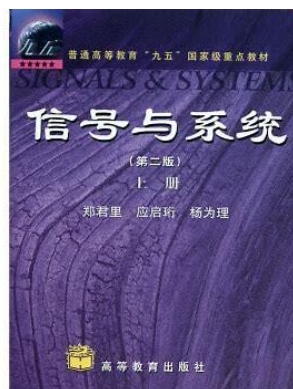
解码(Decoding)

另一种格式
e.g. 字符串



Offset	0	1	2	3	4	5	6	7	-	8	9	A	B	C	D	E	F	ASCII
00000000	FF	D8	FF	E0	00	10	4A	46		49	46	00	01	01	01	00	48	яШяа..JFIF.....H
00000010	00	48	00	00	FF	DB	00	43		00	01	01	01	01	01	01	01	.H...яЫ.С.....
00000020	01	01	01	01	01	01	01	01		01	01	01	01	01	01	01	01	
00000030	01	01	01	01	01	01	01	01		01	01	01	01	01	01	01	01	
00000040	01	01	01	01	01	01	01	01		01	01	01	01	01	01	01	01	
00000050	01	01	01	01	01	01	01	01		01	FF	DB	00	43	01	01	01	яЫ.С...
00000060	01	01	01	01	01	01	01	01		01	01	01	01	01	01	01	01	
00000070	01	01	01	01	01	01	01	01		01	01	01	01	01	01	01	01	
00000080	01	01	01	01	01	01	01	01		01	01	01	01	01	01	01	01	

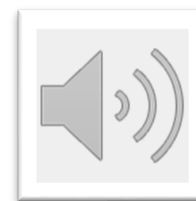
JPEG文件头



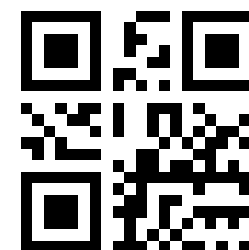
SBN 978-7-04-022559-4



Bar code



Hello world



QR code

为什么用二元编码/二进制？

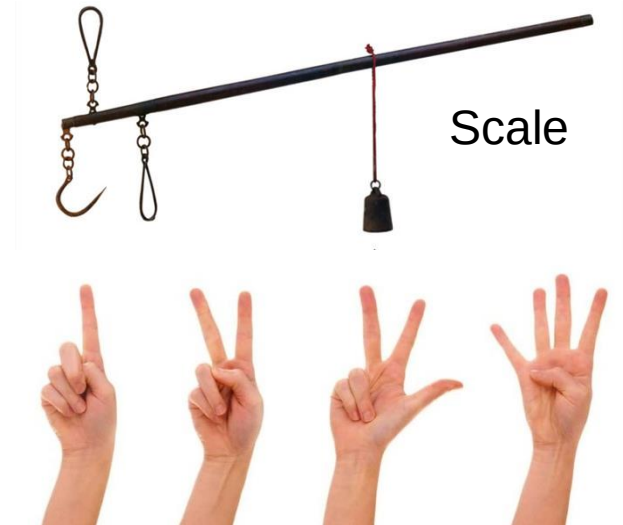
- 容易表示和存储

- 只用两种状态(‘0’ 和 ‘1’)

- 晶体管的导通表示 “1”， 截止表示 “0”
 - 电位的高表示 “1”， 低表示 “0”

- 材料极化特性
- 磁场、自旋方向
-

- 为什么不用十进制？



为什么用二元编码/二进制？

- 运算简单

- 加法和乘法

- $0+0=(0)0$, $0+1=(0)1$, $1+1=(1)0$

- $0\times 0=0$, $0\times 1=0$, $1\times 1=1$

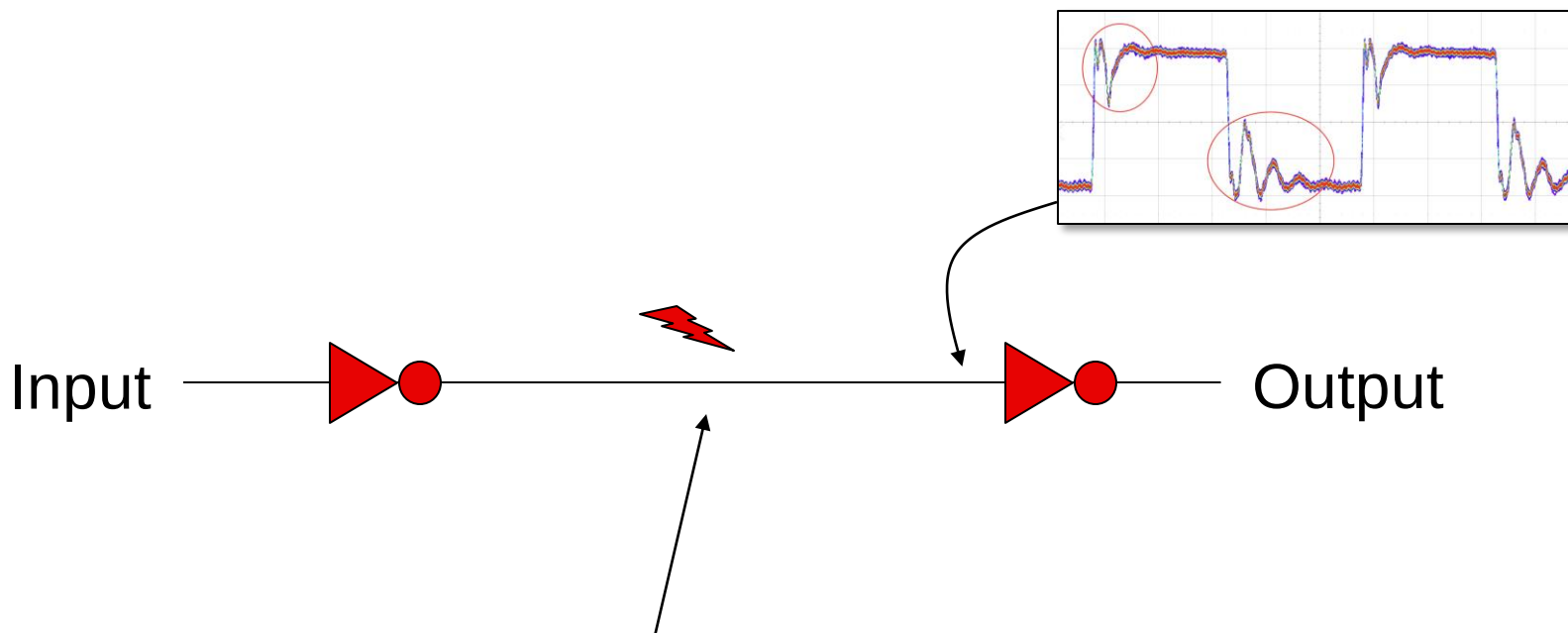
- 逻辑对应：

- 二进制 “1/0” 与逻辑值 “真/假”

为什么用二元编码/二进制？

- 传输简单

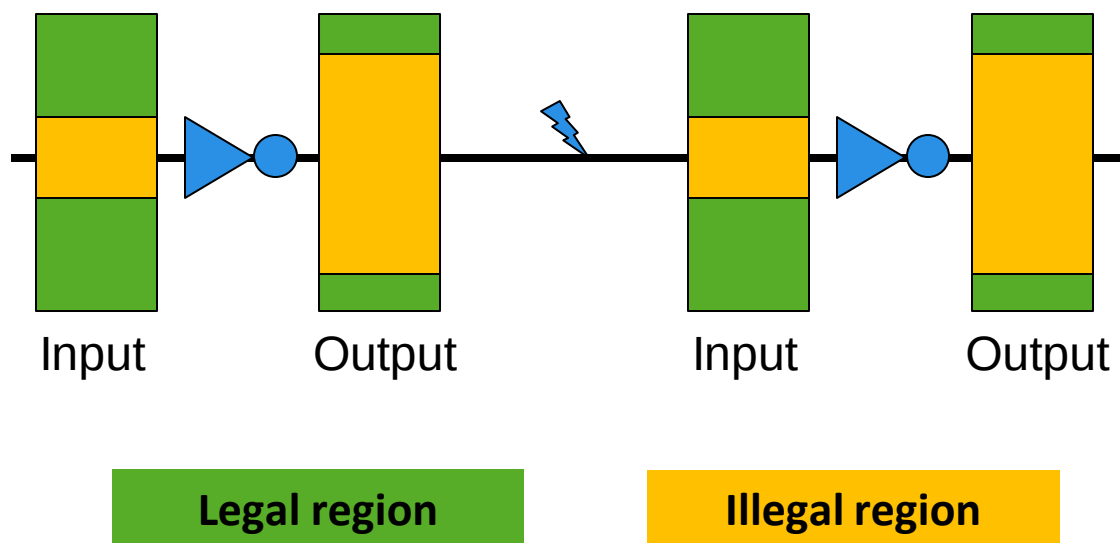
- 噪音容限高 → 可靠性高



问题：如何处理噪音的干扰？

为什么用二元编码/二进制？

- 传输简单
 - 噪音容限高 → 可靠性高



二进制 Binary System

Radix point

MSB (Most Significant Bit)

LSB (Least Significant Bit)

1 0 1 . 1 1

Weight

2^2

2^1

2^0

2^{-1}

2^{-2}

$$B = \sum k_i \times 2^i$$

$k_i: 0, 1$

$$101.11_2 = 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$

$$+ 1 \times 2^{-1} + 1 \times 2^{-2}$$

$$= 5.75_{10}$$

$$= 5 \times 10^1 + 7 \times 10^{-1} + 5 \times 10^{-2}$$

4-bit 二进制整数

BIN				DEC	HEX
2^3	2^2	2^1	2^0		
0	0	0	0	0	0
0	0	0	1	1	1
0	0	1	0	2	2
0	0	1	1	3	3
0	1	0	0	4	4
0	1	0	1	5	5
0	1	1	0	6	6
0	1	1	1	7	7

BIN				DEC	HEX
2^3	2^2	2^1	2^0		
1	0	0	0	8	8
1	0	0	1	9	9
1	0	1	0	10	A
1	0	1	1	11	B
1	1	0	0	12	C
1	1	0	1	13	D
1	1	1	0	14	E
1	1	1	1	15	F

左移与右移操作

Right shift
右移
对应十进制 $\div 2$

1110. 1-bit right shift=0111.0

$$14 \div 2^1 = 7$$

1110. 2-bit right shift=0011.10

$$14 \div 2^2 = 3.5$$

Left shift
左移
对应十进制 $\times 2$

0110. 1-bit left shift=01100.

$$6 \times 2^1 = 12$$

0110. 2-bit left shift=011000.

$$6 \times 2^2 = 16 + 8 = 24$$

另一种十进制表示法？

$$395_{10} = (256+128+8+2+1)_{10} = (0001\ 1000\ 1011)_2$$

$$\left. \begin{array}{l} \bullet\ 3_{10} = 0011_2 \\ \bullet\ 9_{10} = 1001_2 \\ \bullet\ 5_{10} = 0101_2 \end{array} \right\} (395)_{10} \rightarrow (0011\ 1001\ 0101)_{\text{NEW}}$$

Binary-coded decimal (BCD):
每个十进制数字都用二进制单独表示

BCD码

- **Binary-coded decimal**
 - 每个十进制数单独表示
 - 编码方式不唯一
- **8421 BCD码**
 - 有权码
 - 最常用

十进制数	8.4.2.1BCD			
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
权重	8	4	2	1

注意

- 8421BCD和二进制数有区别！
- $(395)_{10} = (0011\ 1001\ 0101)_{\text{BCD}} = (0001\ 1000\ 1011)_2$
 $\neq (0011\ 1001\ 0101)_2$
- $(0011\ 1001\ 0101)_2 = (512+256+128+16+4+1)_{10} = 917_{10}$
- BCD加法
 - $34_{10} + 45_{10} = (0011\ 0100)_{\text{BCD}} + (0100\ 0101)_{\text{BCD}}$
 - 进位 (Carry)
 - Try $(14)_D + (28)_D = (0001\ 0100)_{\text{BCD}} + (0010\ 1000)_{\text{BCD}}$

格雷码

0	0	0
0	0	1
0	1	1
0	1	0
1	1	0
1	1	1
1	0	1
1	0	0

Gray

0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

Binary

- 相邻格雷码两个数之间仅有一位编码不同
- 格雷码将在卡诺图化简中使用

减法、负数

- 加法和减法是最基本的运算

$$\begin{array}{r} 173 \\ +) \quad 44 \\ \hline 217 \end{array}$$

$$\begin{array}{r} 173 \\ -) \quad 46 \\ \hline 127 \end{array}$$

$$\begin{array}{r} 10101101 \\ +) \quad 00101100 \\ \hline 11011001 \end{array}$$

$$\begin{array}{r} 010100 \\ 10101101 \\ -) \quad 00101110 \\ \hline 01111111 \end{array}$$

- 数有正有负，如何表示负数？

数值空间的划分

- **n bit二进制数值空间可表示 2^n 个数值**
 - 000, 001, 010, 011,
100, 101, 110, 111
 - 哪些表示正数, 哪些表示负数?
 - 划分标准如何?

负数的表示方式

- 一种方式：有符号原码

$$01010101_2 = +85_{10}$$

$$11010101_2 = -85_{10}$$

Problem??

$$+85_{10} + -85_{10} = 0_{10}$$
$$01010101_2 + 11010101_2 \neq 00000000_2$$

Signed binary code

+0 (Decimal)	+1	+2	+3	+4	+5	+6	+7
0000	0001	0010	0011	0100	0101	0110	0111
-0 (Decimal)	-1	-2	-3	-4	-5	-6	-7
1000	1001	1010	1011	1100	1101	1110	1111

- 如何保证 $X+(-X)=0$?
- 2-补码（简称补码，不同于1-补码！）
 - 为使 $X+(-X)=0$ ，补码如何表示？
 - 翻转（+85）的每bit并加1，得到（-85）

补码计算

$$+85_{10} + -85_{10} = 0_{10}$$
$$01010101_2 + \text{xxxxxxxx}_2 = 00000000_2$$
$$10101011_2$$

比较

Signed binary code (原码)

+0 (Decimal)	+1	+2	+3	+4	+5	+6	+7
0000	0001	0010	0011	0100	0101	0110	0111
-0 (Decimal)	-1	-2	-3	-4	-5	-6	-7
1000	1001	1010	1011	1100	1101	1110	1111

2's complement binary code (2-补码)

0 (Decimal)	+1	+2	+3	+4	+5	+6	+7	
0000	0001	0010	0011	0100	0101	0110	0111	
	-1	-2	-3	-4	-5	-6	-7	-8
	1111	1110	1101	1100	1011	1010	1001	1000

- ❖ Range: $[-2^{n-1}, 2^{n-1}-1]$; Only one representation for 0
- ❖ MSB is the sign bit (**MSB=0, positive; MSB=1, negative.**)

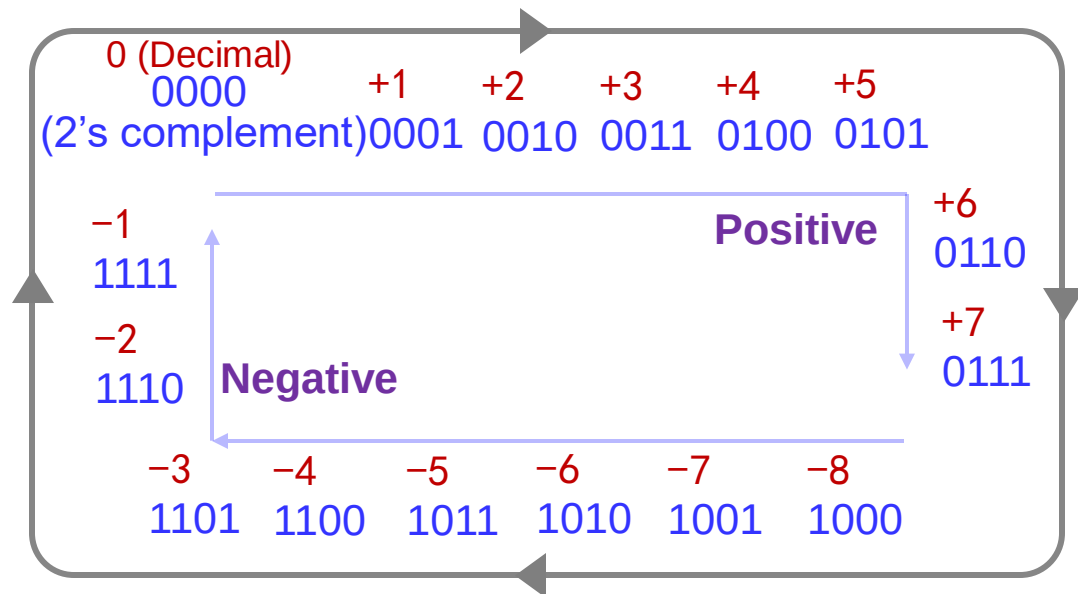
理解2-补码

- 十进制负数和2-补码的转化方式？举两个例子
 - Q: $(-3)_{10}$ 如何转化为 101_2 ?
 - 写出 3_{10} 的二补码 011_2 , 翻转每个bit得到 100_2 , 并加一得到 101_2
 - Q: 101_2 如何转化为 $(-3)_{10}$? 一般地, 对于n位负数2-补码 $(k_n k_{n-1} \dots k_1)_2$ ($k_n=1$), 如何转化为 $(-N)_{10}$?
 - 把上一问的方法逆向使用, 或者:
 - 把 $(-N)_{10}$ 的2-补码理解为无符号数 N_{us} :
$$N_{us} + N = (11\dots1)_2 + 1 = 2^n;$$
$$\rightarrow (-N) = N_{us} - 2^n = k_n 2^{n-1} + k_{n-1} 2^{n-2} + \dots + k_2 2^1 + k_1 - 2^n$$
$$= -2^{n-1} + k_{n-1} 2^{n-2} + \dots + k_2 2^1 + k_1$$

理解2-补码

- 使用无符号二进制的加/减逻辑
 - 把 $(1111)_2$ 加1, 可以得到什么?
 - 把 $(0000)_2$ 减1, 可以得到什么?
 - 因此, 把1111当做-1可以复用加法电路
- 在此范围内的所有加减操作都是有效的!

从正数到负数:
翻转每个比特并
加一



补码的应用：用加法来实现减法！

- ‘-N’的补码

- 无符号数值： $N_{us} = 2^n - N$

- 减法 (3-bit 例子)

- $2 - 1 = 2 + 2^3 - 1 = 2 + (2^3 - 1) = 2 + (111)_{us}$

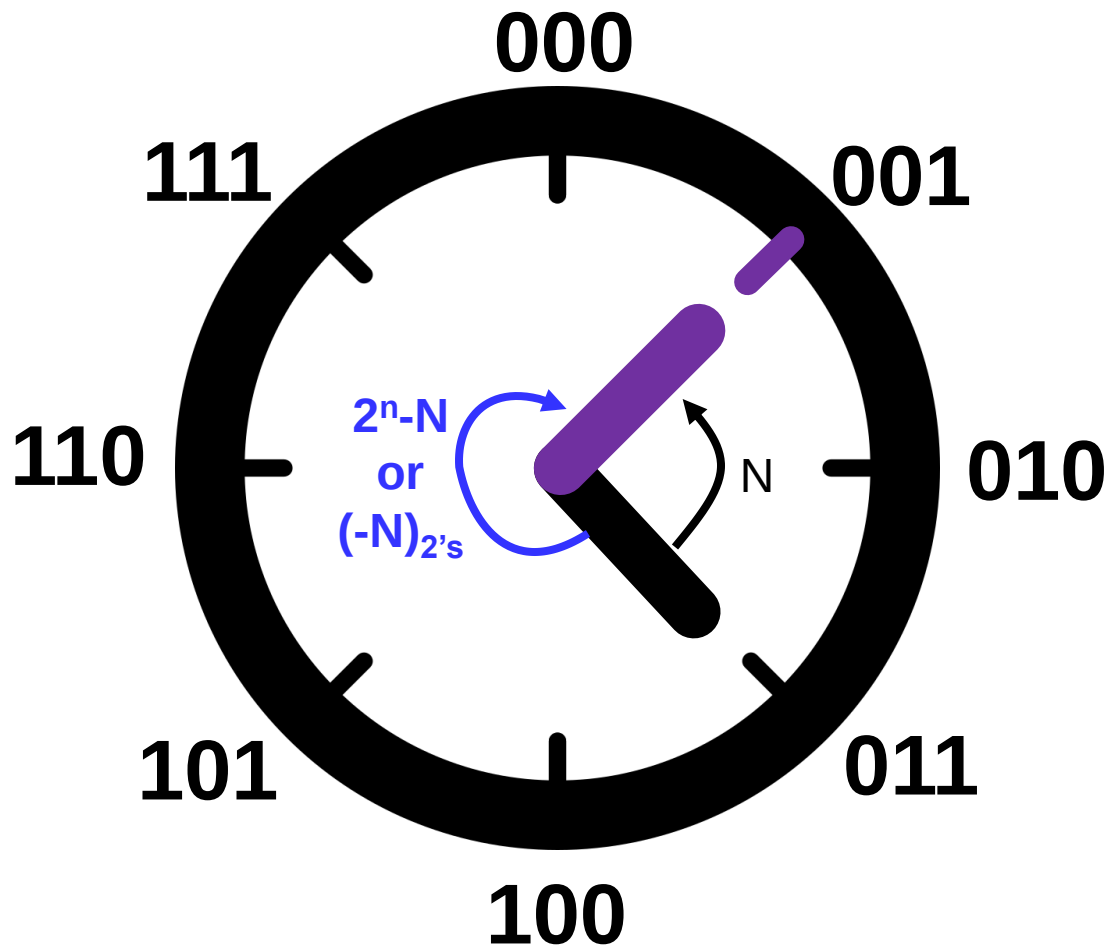
- $A - N = A + (-N) = A + (2^n - N) = A + N_{us}$

010	↔	1010
<u>- 001</u>		<u>- 0001</u>
001		1001

	↔	010
		<u>+ 111</u>
		1001

减法与调时钟

$$A - N = A + (-N) = A + (2^n - N) = A + (-N)_{2's}$$



011-010

VS

011+110

用加法来实现减法

- $A - B = A + (-B) = A + (0 - B)$

例：六位二进制数的计算：8-20→8+(-20)

传统减法模式：

$$\begin{array}{r} 1010 \\ 8: 001000 \\ -) 20: 010100 \\ \hline -12: 110100 \end{array}$$

补码减法模式：

$$\begin{array}{r} 001000 \\ +) 101100 \\ \hline 110100 \end{array}$$

实例验证，补码可以利用加法器实现完整的加减法！

可有问题？

4-bit digits

$$\begin{array}{r}
 +3 \quad 0011 \\
 +) \quad +4 \quad +) \quad 0100 \\
 \hline
 +7 \quad 0111
 \end{array}$$

$$\begin{array}{r}
 +6 \quad 0110 \\
 +) \quad -3 \quad +) \quad 1101 \\
 \hline
 +3 \quad 10011
 \end{array}$$

Ignore

$$\begin{array}{r}
 +5 \quad 0101 \\
 +) \quad +6 \quad +) \quad 0110 \\
 \hline
 +11 \quad 1011
 \end{array}$$

$$\begin{array}{r}
 -6 \quad 1010 \\
 +) \quad -3 \quad +) \quad 1101 \\
 \hline
 -9 \quad 10111
 \end{array}$$

Ignore

-5

+7

溢出

- 什么是溢出
 - 计算结果超出了可表示数的范围
 - $-2^{n-1} \sim 2^{n-1}-1$
- 如何判断溢出（什么时候可能溢出）
 - 两正数相加，结果变负数（或零）
 - 两负数相加，结果变正数（或零）

• 例：(1) -96-33

溢出

	1	0	1	0	0	0	0	0
+	1	1	0	1	1	1	1	1
1	0	1	1	1	1	1	1	1

127

(2) -96-32

	1	0	1	0	0	0	0	0
+	1	1	1	0	0	0	0	0
1	1	0	0	0	0	0	0	0

-128

未溢出

处理溢出

- 增加位数

			+6		00110
			+) -3		+) 11101
			+3		100011
					↑ 忽略
+5		00101		-6	11010
+) +6	+) 00110		+) -3	+) 11101	
+11	01011		-9	110111	
				↑ 忽略	

处理溢出

- 增加位数
 - 正数增加的位数补0
 - $0110 \rightarrow 00110$
 - $2^2 + 2^1 \rightarrow 2^2 + 2^1$
 - 负数增加的位数补1
 - $1101 \rightarrow 11101$
 - $-2^3 + 2^2 + 2^0 \rightarrow -2^4 + 2^3 + 2^2 + 2^0$
 - 负数原来的负权重位变成正权重位

溢出的简单判断

- 判断是否超出数值范围
- 对符号位的进位输入(Cin)和输出(Cout)做异或

Cin	Cout	结果
0	0	正确
1	0	溢出，上溢（正数相加变负数）
0	1	溢出，下溢（负数相加变正数）
1	1	正确

符号位 最高位
无进位 有进位

↓ ↓

$$\begin{array}{r} 01000000 \\ + 01000000 \\ \hline 01000000 \end{array}$$

-128
上溢

两个正数相加变负数

符号位 最高位
有进位 无进位

↓ ↓

$$\begin{array}{r} 10100000 \\ + 11011111 \\ \hline 10111111 \end{array}$$

127
下溢

两个负数相加变正数

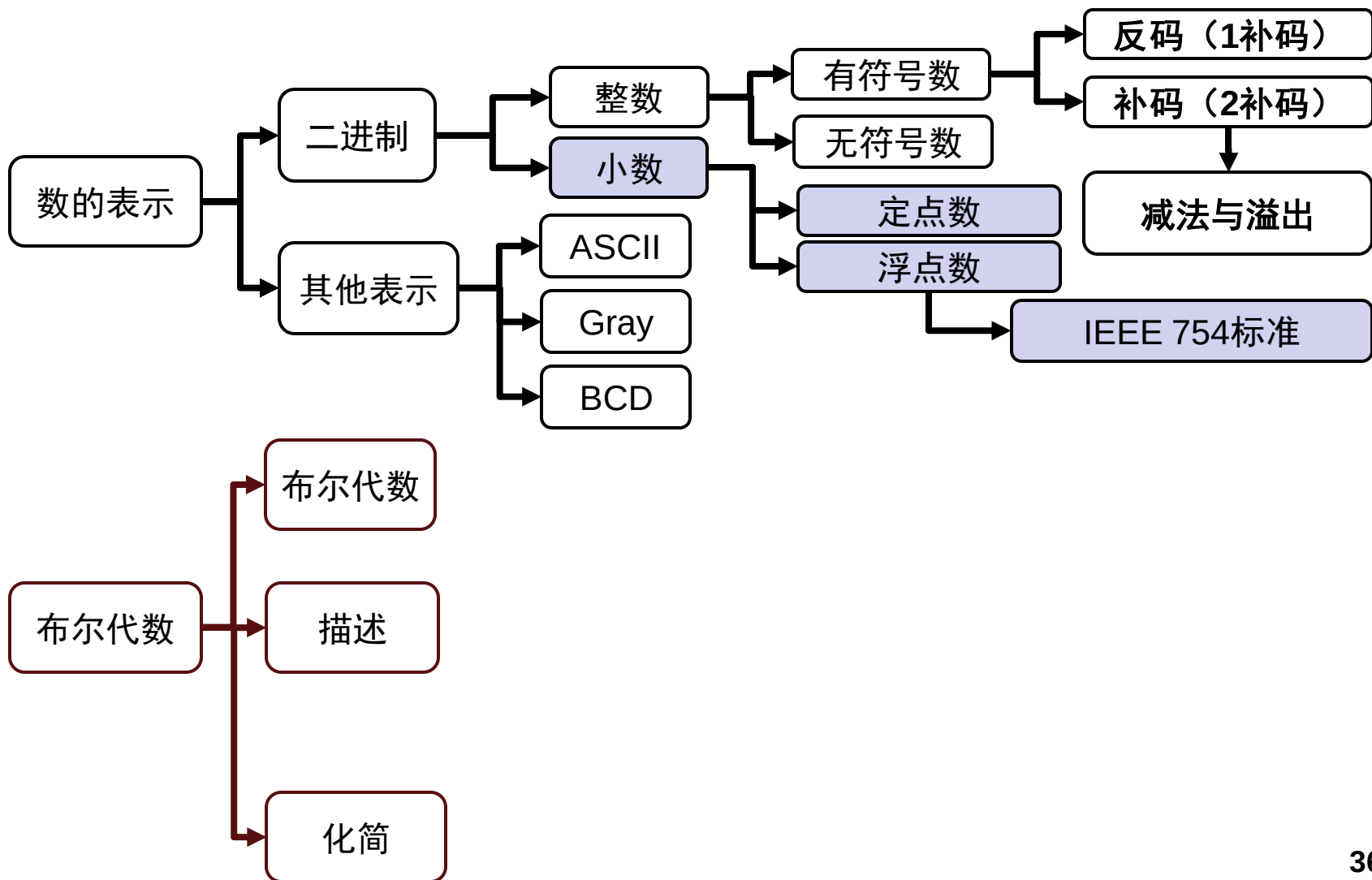
符号位 最高位
有进位 有进位

↓ ↓

$$\begin{array}{r} 10100000 \\ + 11100000 \\ \hline 11000000 \end{array}$$

-128
未溢出

数的表示与布尔代数



本节课目录

- 数的编码与二进制表示
- 算术与函数的布尔代数表示
- 布尔表达式的化简

布尔代数的定义

- 布尔代数是数字系统的数学基础
- 用逻辑运算符表示输入逻辑变量与输出逻辑变量之间的关系
 - 运算: NOT (非), AND (与), OR (或)
 - 优先权: 非最高, 与次之, 或最低
 - 运算符: $+$ (或/逻辑加), $\bullet$ (与/逻辑乘), $'$ (—) (非/反)

基本定律 (1)

- **Commutative Law 交换律**
 - $X+Y=Y+X$ $XY=YX$
- **Associative Law 结合律**
 - $(X+Y)+Z=X+(Y+Z)=X+Y+Z$
 - $(XY)Z=X(YZ)=XYZ$
- **Distributive Law 分配律**
 - $X+(Y \cdot Z)=(X+Y)(X+Z)$
 - $X(Y+Z)=XY+XZ$
- **Complement 互补律**
 - $X+X'=1$ $X \cdot X'=0$

基本定律 (2)

- Absorption 吸收率
 - $X + XY = X$
 - (可推出 $X + X'Y = X + Y$)
- Idempotent 幂等律
 - $X + X = X$; $X \cdot X = X$
- Involution 对合
 - $(X')' = X$
- 与0 & 1的运算
 - $X + 0 = X$, $X + 1 = 1$
 - $X \cdot 1 = X$, $X \cdot 0 = 0$

基本定理 (3)

- Consensus theorem 一致性定理（多数定理）
 - $(X+Y)(Y+Z)(X'+Z)=(X+Y)(X'+Z)$
 - $X \cdot Y + X' \cdot Z + Y \cdot Z = XY + X'Z$
 - 用于化简逻辑表达式（代数化简法用到）
 - 用于消除组合电路的竞争冒险（下一次课用到）

布尔函数

- 多种表现形式
 - 布尔代数的表达式: $f(X_1, X_2, \dots, X_n, 1, 0, \cdot, +)$
 - Variables and operators
 - 真值表 (Truth table)
 - 电路图 (Circuit diagram)

德摩根律(De Morgan's Law)

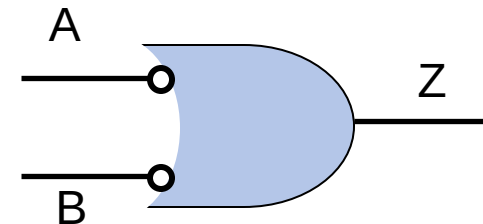
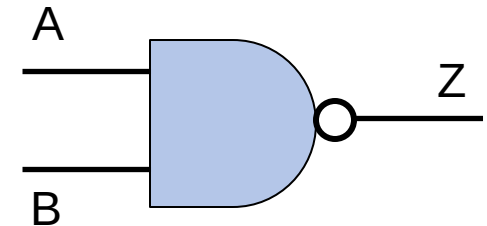
- 函数反演的一般形式：
 - $\{f(X_1, \dots, X_n, 0, 1, +, \cdot)\}' = f(X_1', \dots, X_n', 1, 0, \cdot, +)$
 - $\text{AND} \leftrightarrow \text{OR}, 0 \leftrightarrow 1, X \leftrightarrow X'$
- 例子
 - $\overline{AB} = \overline{A} + \overline{B}$
 - $\overline{A + B} = \overline{A} \cdot \overline{B}$
 - 其他

德摩根律(De Morgan's Law)

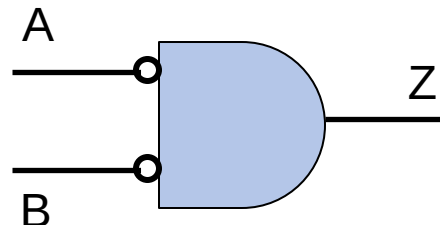
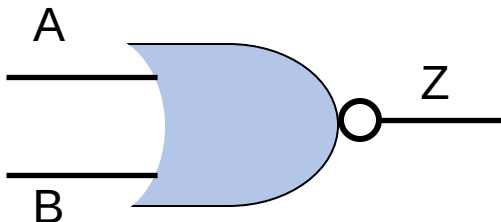
- $(X+Y+Z+\dots)' = X' \cdot Y' \cdot Z' \cdot \dots$
- $(X \cdot Y \cdot Z \cdot \dots)' = X' + Y' + Z' + \dots$

$$\overline{AB} = \overline{A} + \overline{B}$$

NAND gate



$$\overline{A + B} = \overline{A} \cdot \overline{B}$$



NOR gate

用真值表验证德摩根定律

X	Y	X'	Y'	$X+Y$	XY	$(X+Y)'$	$X'Y'$	$(XY)'$	$X'+Y'$
0	0	1	1	0	0	1	1	1	1
0	1	1	0	1	0	0	0	1	1
1	0	0	1	1	0	0	0	1	1
1	1	0	0	1	1	0	0	0	0

求反

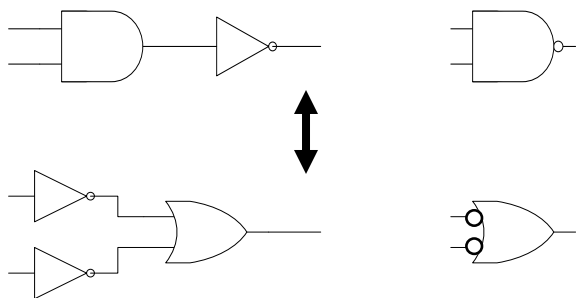
$$Y = A(B + C) + CD$$

- 与和或互换
- 逻辑0和1互换
- 布尔变量求反

$$\begin{aligned} Y' &= [A(B + C) + CD]' \\ &= [A(B + C)]' \bullet (CD)' \\ &= [A' + (B + C)'] \bullet (C' + D') \\ &= (A' + B'C') \bullet (C' + D') \end{aligned}$$

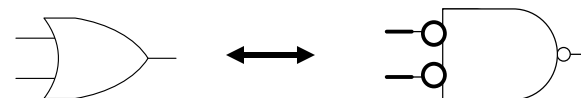
用德摩根定律变换逻辑门电路的形式

$$\overline{A \cdot B} = \bar{A} + \bar{B}$$



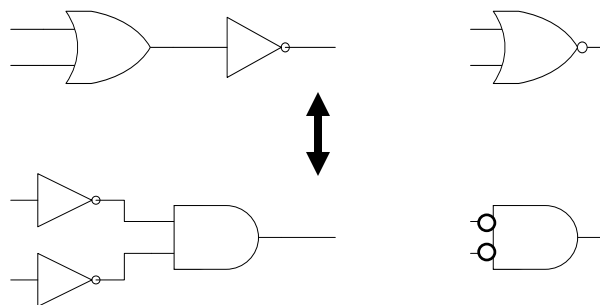
与非门等价于对输入取反的或门

$$A + B = \overline{\bar{A} \cdot \bar{B}}$$



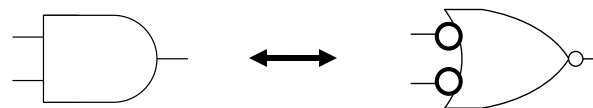
或门等价于对输入取反的与非门

$$\overline{A + B} = \bar{A} \cdot \bar{B}$$



或非门等价于对输入取反的与门

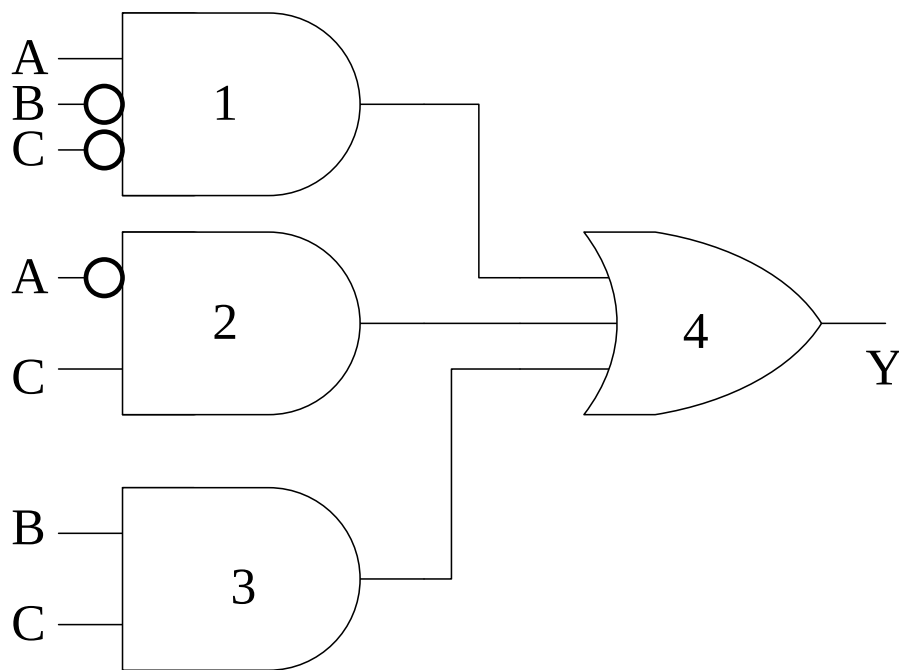
$$A \cdot B = \overline{\bar{A} + \bar{B}}$$



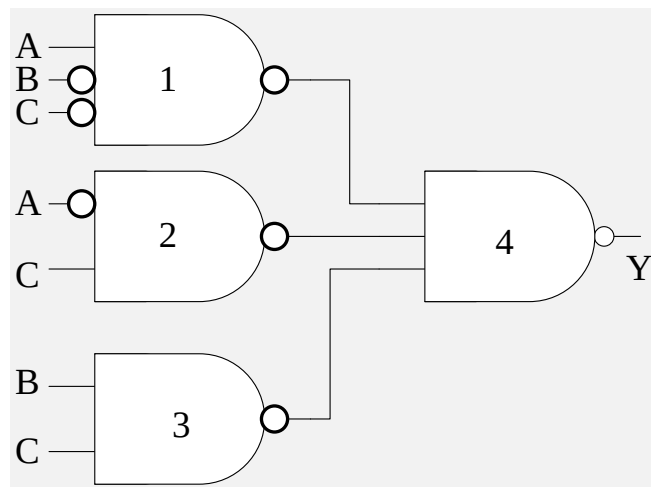
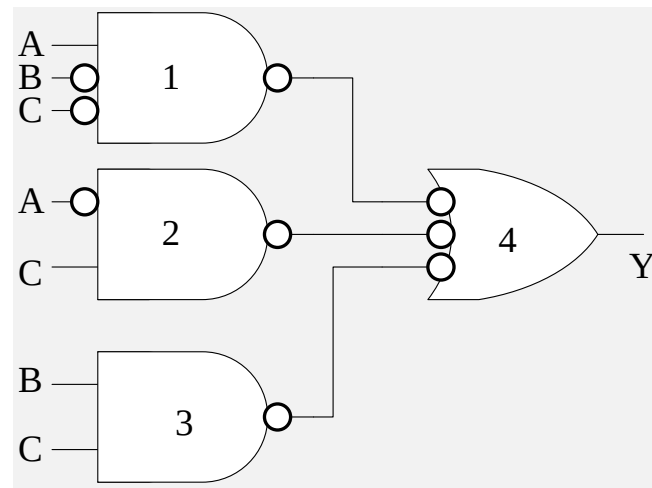
与门等价于对输入取反的或非门

将与或表达式变换成仅用与非门实现

$$Y = AB'C' + A'C + BC$$



$$Y = [(AB'C')'(A'C)'(BC)']'$$



其他布尔函数

- 与

$0 \cap 0 = 0$
 $0 \cap 1 = 0$
 $1 \cap 0 = 0$
 $1 \cap 1 = 1$

- 或

$0 \cup 0 = 0$
 $0 \cup 1 = 1$
 $1 \cup 0 = 1$
 $1 \cup 1 = 1$

- 非

$0' = 1$
 $1' = 0$

- 乘法 $a \times b = f$

$0 \times 0 = 0$
 $0 \times 1 = 0$
 $1 \times 0 = 0$
 $1 \times 1 = 1$

- 加法 $a + b = cs$

$0 + 0 = 00$
 $0 + 1 = 01$
 $1 + 0 = 01$
 $1 + 1 = 10$

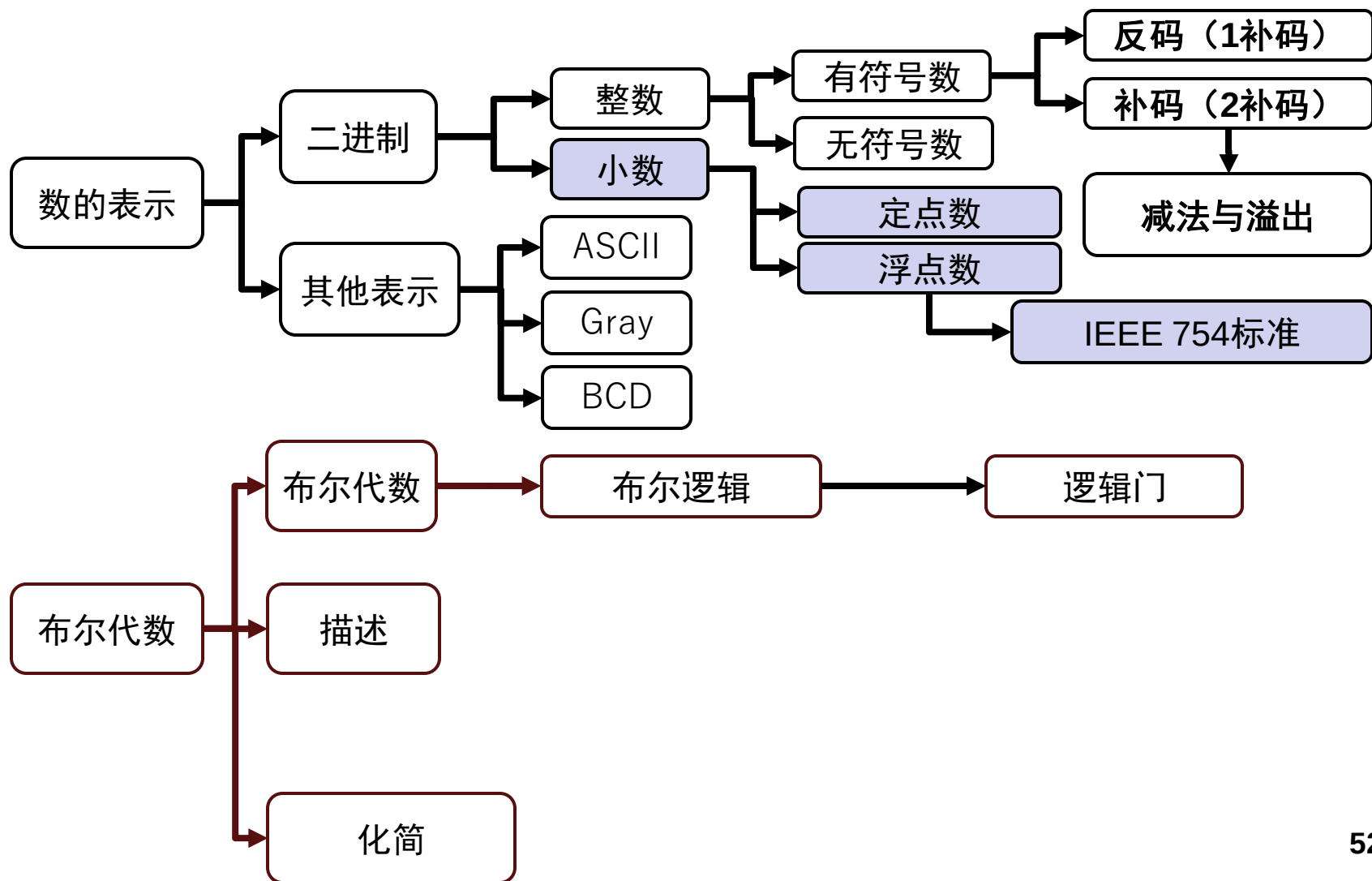
与=乘

与=进位

异或=和
异或: $a \oplus b = (\sim a \cap b) \cup (a \cap \sim b)$

- $f = a \bullet b$
- $c = a \bullet b$
- $s = ab' + a'b = a \oplus b$

数的表示与布尔代数



函数的布尔代数描述

- 二元域上函数的表示方法：
 - 真值表（及卡诺图）
 - 表达式
 - 其他方式（程序、
Binary decision diagram - BDD树）
- 例：日历子系统
 - 根据月份和是否闰年的标志，给出这个月份的天数

日历子系统的C程序

```
Integer number_of_days (month, leap_flag) {  
    switch (month){  
        case 'january': return (31);  
        case 'february': if (leap_flag ==1) then return (29) else return  
            (28);  
        case 'march': return (31);  
        case 'april': return (30);  
        case 'may': return (31);  
        case 'june': return (30);  
        case 'july': return (31);  
        case 'august': return (31);  
        case 'september': return (30);  
        case 'october': return (31);  
        case 'november': return (30);  
        case 'december': return (31);  
        default: return (0);  
    }  
}
```


日历子系统的逻辑实现

- 组合逻辑实现过程
 - 逻辑抽象
 - 输入: $\text{month}(a_8, a_4, a_2, a_1)$, leap_flag
 - 输出: d28, d29, d30, d31
 - 逻辑函数: 真值表
 - 逻辑表达式与化简
 - 逻辑电路

真值表—输出与输入之间的逻辑关系

$a_8a_4a_2a_1$

无关项

- 输出不随输入变化
- 实际情况中不存在
- 不影响功能

Month	Leap	d28	d29	d30	d31
0000	-	-	-	-	-
0001	-	0	0	0	1
0010	0	1	0	0	0
0011	1	0	1	0	0
0100	-	0	0	0	1
0101	-	0	0	1	0
0110	-	0	0	0	1
0111	-	0	0	1	0
1000	-	0	0	0	1
1001	-	0	0	1	0
1010	-	0	0	0	1
1011	-	0	0	1	0
1100	-	0	0	0	1
1101	-	-	-	-	-
111-	-	-	-	-	-

逻辑函数

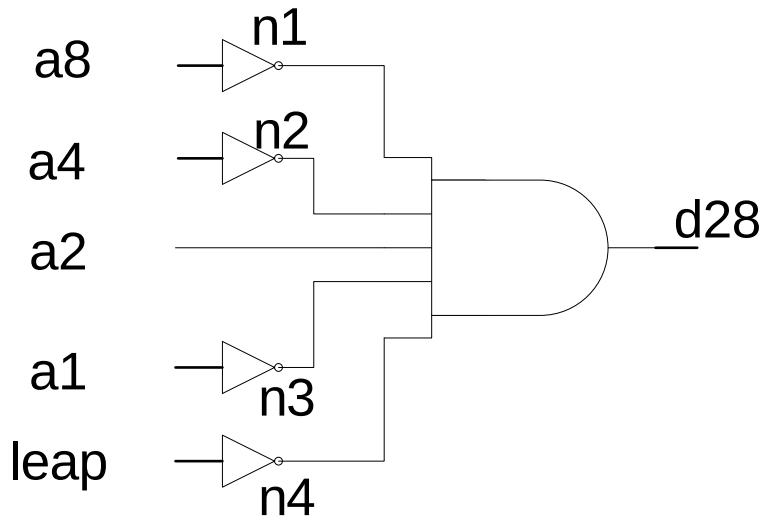
- **d28** = a_8' and a_4' and a_2 and a_1' and leap'
- **d29** = a_8' and a_4' and a_2 and a_1' and leap

$a_8 a_4 a_2 a_1$ →

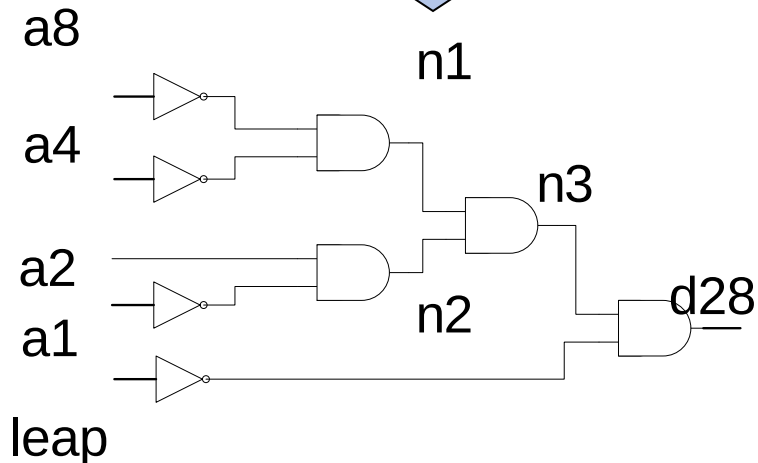
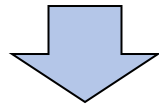
Month	Leap	d28	d29
0000	-	-	-
0001	-	0	0
0010	0	1	0
0010	1	0	1

d28的逻辑电路

Verilog: 一种硬件描述语言



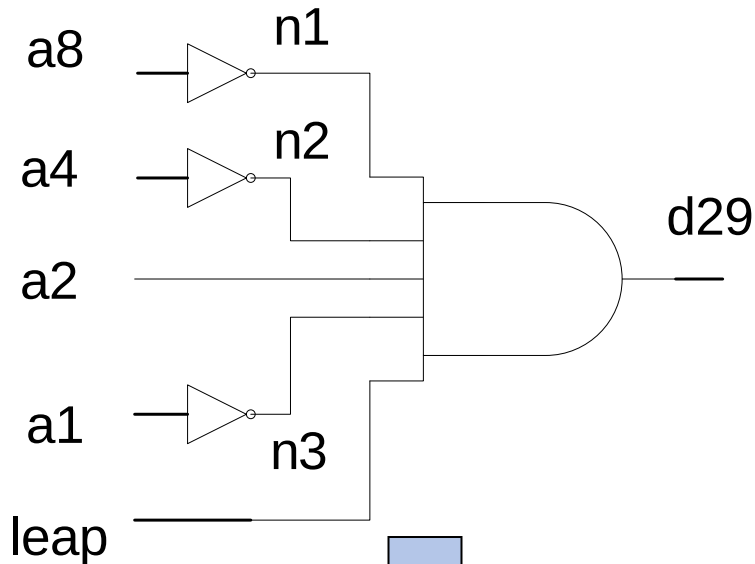
```
module CalD28 (a8,a4,a2,a1,leap,d28);  
  input  a8,a4,a2,a1,leap;  
  output d28;  
  not u1(n1, a8);  
  not u2(n2, a4);  
  not u3(n3, a1);  
  not u4(n4, leap);  
  and u5(d28, n1, n2, a2, n3, n4);  
endmodule
```



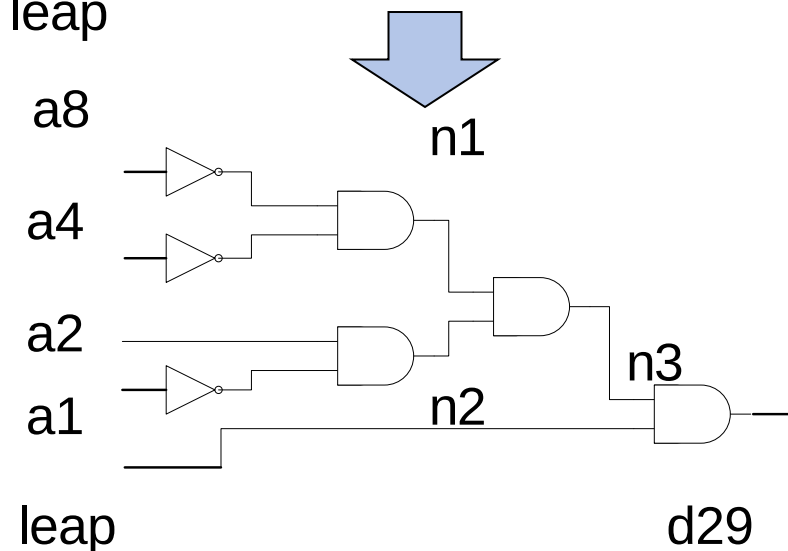
```
module CalD28 (a8,a4,a2,a1,leap,d28);  
  input  a8,a4,a2,a1,leap;  
  output d28;  
  wire n1,n2,n3;  
  assign n1=(~a8)&(~a4);  
  assign n2=a2&(~a1);  
  assign n3=n1&n2;  
  assign d28=n3&(~leap);  
endmodule
```

d29的逻辑电路

Verilog代码



```
module CalD29 (a8,a4,a2,a1,leap,d29);
    input  a8,a4,a2,a1,leap;
    output d29;
    not u1(n1, a8);
    not u2(n2, a4);
    not u3(n3, a1);
    and u4(d29, n1, n2, a2, n3, leap);
endmodule
```



```
module CalD29 (a8,a4,a2,a1,leap,d29);
    input  a8,a4,a2,a1,leap;
    output d29;
    wire n1,n2,n3;
    assign n1=(~a8)&(~a4);
    assign n2=a2&(~a1);
    assign n3=n1&n2;
    assign d28=n3&leap;
endmodule
```

d30和d31呢？

- $d30 = a_8' \cdot a_4 \cdot a_2' \cdot a_1' \text{ or } a_8' \cdot a_4 \cdot a_2 \cdot a_1' \text{ or } a_8 \cdot a_4' \cdot a_2' \cdot a_1 \text{ or } a_8 \cdot a_4' \cdot a_2 \cdot a_1$
- $d31 = a_8' \cdot a_4' \cdot a_2' \cdot a_1 \text{ or } a_8' \cdot a_4' \cdot a_2 \cdot a_1 \text{ or } a_8' \cdot a_4 \cdot a_2' \cdot a_1 \text{ or } a_8' \cdot a_4 \cdot a_2 \cdot a_1 \text{ or } a_8 \cdot a_4' \cdot a_2' \cdot a_1' \text{ or } a_8 \cdot a_4' \cdot a_2 \cdot a_1' \text{ or } a_8 \cdot a_4 \cdot a_2' \cdot a_1' \text{ or } a_8 \cdot a_4 \cdot a_2 \cdot a_1'$

考虑电路，

如何表示它们对应的布尔代数式？

——逻辑化简

Month	d30	d31
0000	-	-
0001	0	1
0010	0	0
0011	0	1
0100	1	0
0101	0	1
0110	1	0
0111	0	1
1000	0	1
1001	1	0
1010	0	1
1011	1	0
1100	0	1
1101	-	-
111-	-	-

布尔代数式的化简

- 两级逻辑
 - 规范形式
 - 卡诺图
- 两级逻辑的化简
 - 卡诺图化简法
 - Q-M 法

标准形式：两级逻辑

- SOP 和 POS

- Sum of Products 与或表达式

$$f(A, B, C, D) = AB + A\bar{C} + C\bar{D}$$

- Product of Sums 或与表达式

$$f(A, B, C, D) = (A + B + C)(\bar{C} + \bar{D})$$

$$\begin{aligned} d31 = & a_8' \cdot a_4' \cdot a_2' \cdot a_1 \text{ or } a_8' \cdot a_4' \cdot a_2 \cdot a_1 \\ & \text{or } a_8' \cdot a_4 \cdot a_2' \cdot a_1 \text{ or } a_8' \cdot a_4 \cdot a_2 \cdot a_1 \text{ or } \\ & a_8 \cdot a_4' \cdot a_2' \cdot a_1' \text{ or } a_8 \cdot a_4' \cdot a_2 \cdot a_1' \text{ or } \\ & a_8 \cdot a_4 \cdot a_2' \cdot a_1' \end{aligned} \rightarrow \Sigma(ABCD)$$

Month	d30	d31
0000	-	-
0001	0	1
0010	0	0
0011	0	1
0100	1	0
0101	0	1
0110	1	0
0111	0	1
1000	0	1
1001	1	0
1010	0	1
1011	1	0
1100	0	1
1101	-	-
111-	-	-

逻辑函数的最小项，记为m

在n变量逻辑函数中，包含全部n个变量的与项，其中每个变量必须且只以原变量或反变量的形式出现一次

Minterm	m_i	Logic values of variables			Subscript i
		A	B	C	
$A'B'C'$	m_0	0	0	0	0
$A'B'C$	m_1	0	0	1	1
$A'BC'$	m_2	0	1	0	2
$A'BC$	m_3	0	1	1	3
$AB'C'$	m_4	1	0	0	4
$AB'C$	m_5	1	0	1	5
ABC'	m_6	1	1	0	6
ABC	m_7	1	1	1	7

Weight

2^2

2^1

2^0

MSB

LSB

用最小项表示逻辑函数(SOP)

$$Y = f(A, B, C) = \bar{A}B + BC + A\bar{B}\bar{C}$$

$$= \bar{A}B(C + \bar{C}) + (A + \bar{A})BC + A\bar{B}\bar{C}$$

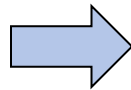
$$\begin{array}{ccccccc} \text{MSB} \nearrow & \bar{A}BC & + & \bar{A}B\bar{C} & + & ABC & + & \bar{A}BC & + & A\bar{B}\bar{C} \nwarrow \text{LSB} \\ & \downarrow & & \downarrow & & \downarrow & & \downarrow & & \\ 011_2 = 3_{10} & & 010_2 = 2_{10} & & 111_2 = 7_{10} & & 100_2 = 4_{10} \end{array}$$

$$= m_3 + m_2 + m_7 + m_3 + m_4$$

$$= \sum m(2, 3, 4, 7)$$

在真值表中使用最小项

$a_8a_4a_2a_1$	d30
0000	-
0001	0
0010	0
0011	0
0100	1
0101	0
0110	1
0111	0
1000	0
1001	1
1010	0
1011	1
1100	0
1101	-
111-	-



$a_8a_4a_2a_1$	Min-term	d30
0000	m0	-
0001	m1	0
0010	m2	0
0011	m3	0
0100	m4	1
0101	m5	0
0110	m6	1
0111	m7	0
1000	m8	0
1001	m9	1
1010	m10	0
1011	m11	1
1100	m12	0
1101	m13	-
111-	m14-m15	-

POS逻辑表示

$$\begin{aligned} Y &= (A + B) \cdot (\bar{A} + B + C) \\ &= (A + B + C \cdot \bar{C}) \cdot (\bar{A} + B + C) \\ &= (A + B + C)(A + B + \bar{C})(\bar{A} + B + C) \\ &= M_0 M_1 M_4 \\ &= \prod M(0,1,4) \end{aligned}$$

逻辑函数的最大项，记为M

在n变量逻辑函数中，包含全部n个变量的或项，其中每个变量必须且只以原变量或反变量的形式出现一次

Max-term	M_i	Logic values of variables			Subscript i
		A	B	C	
$A+B+C$	M_0	0	0	0	0
$A+B+C'$	M_1	0	0	1	1
$A+B'+C$	M_2	0	1	0	2
$A+B'+C'$	M_3	0	1	1	3
$A'+B+C$	M_4	1	0	0	4
$A'+B+C'$	M_5	1	0	1	5
$A'+B'+C$	M_6	1	1	0	6
$A'+B'+C'$	M_7	1	1	1	7

Weight

2^2

MSB

2^1

2^0

LSB

最小项 m_i & 最大项 M_i

规律: $m_6 = ABC\bar{C} = \overline{\overline{A}\overline{B}\overline{C}} = \overline{\overline{A} + \overline{B} + \overline{C}} = \overline{M_6}$

$$Y = \sum m(2, 3, 4, 7) \longrightarrow \bar{Y} = \overline{m_2 + m_3 + m_4 + m_7}$$

为什么?

$$\begin{aligned}\bar{Y} &= \overline{m_2 + m_3 + m_4 + m_7} \\ &= \overline{M_2 + M_3 + M_4 + M_7} \\ &= \overline{\overline{M_2} \bullet \overline{M_3} \bullet \overline{M_4} \bullet \overline{M_7}} \\ &= M_2 \cdot M_3 \cdot M_4 \cdot M_7 \\ &= \prod M(2, 3, 4, 7)\end{aligned}$$

最小项 m_i & 最大项 M_i

规律: $m_6 = ABC\bar{C} = \overline{\overline{A}\overline{B}\overline{C}} = \overline{\overline{A} + \overline{B} + \overline{C}} = \overline{M_6}$

$$Y = \sum m(2, 3, 7)$$



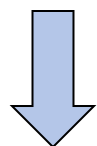
$$\bar{Y} = \overline{m_2 + m_3 + m_7}$$

$$= \overline{\overline{M_2} + \overline{M_3} + \overline{M_7}}$$

$$= \overline{\overline{M_2}} \cdot \overline{\overline{M_3}} \cdot \overline{\overline{M_7}}$$

$$= M_2 \cdot M_3 \cdot M_7$$

$$= \prod M(2, 3, 7)$$



$$\bar{Y} = \overline{m_2 + m_3 + m_7}$$

$$= m_0 + m_1 + m_4 + m_5 + m_6$$

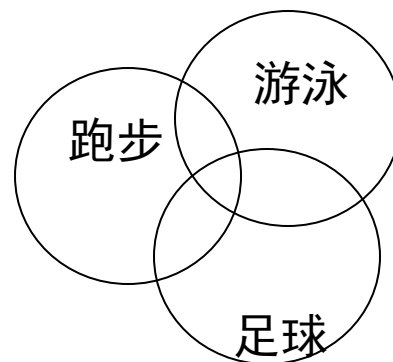
$$= \sum m(0, 1, 4, 5, 6)$$

$$\sum m(0, 1, 4, 5, 6) = \prod M(2, 3, 7)$$

卡诺图

• 逻辑表示图的发展

- 文氏图，1881年，
表示集合关系
- 维奇图(Veitch Diagram)，
1952年，爱德华·维奇提出后
用以描述并最小化布尔表达式
- 卡诺图，1953年，美国贝尔实
验室的电信工程师莫里斯·卡
诺(Maurice Karnaugh)在
1953年根据维奇图改进。



		A	$\bar{A}$	
				$\bar{C}$
B	12	13	5	4
	14	1 ₁₅	1 ₇	6
$\bar{B}$	10	1 ₁₁	1 ₃	2
	1 ₈	1 ₉	1	0
	$\bar{D}$	D	$\bar{D}$	$\bar{C}$

AB \ CD	00	01	11	10
00	1	0	0	1
01	1	0	*	*
11	1	0	1	1
10	1	0	1	1

$$Z = \bar{D} + A \cdot C$$

两变量的卡诺图

$f(A, B)$

	B	0	1
A			
0			
1			

	B	0	1
A			
0		$m_0(A'B')$	$m_1(A'B)$
1		$m_2(AB')$	$m_3(AB)$

例如： $f(A,B)=AB'$

	B	0	1
A			
0		0	0
1		1	0

三变量的卡诺图

BC	00	01	11	10
A	←格雷码的应用→			
0				
1				

$$f(A, B, C)$$

BC	00		01		11		10	
A								
0	$m_0(A'B'C')$		$m_1(A'B'C)$		$m_3(A'BC)$		$m_2(A'BC')$	
1	$m_4(AB'C')$		$m_5(AB'C)$		$m_7(ABC)$		$m_6(ABC')$	

四变量卡诺图的一种形式

$$f(A, B, C, D)$$

CD AB	00	01	11	10
00	$m_0(A'B'C'D')$	$m_1(A'B'C'D)$	$m_3(A'B'CD)$	$m_2(A'B'CD')$
01	$m_4(A'BC'D')$	$m_5(A'BC'D)$	$m_7(A'BCD)$	$m_6(A'BCD')$
11	$m_{12}(ABC'D')$	$m_{13}(ABC'D)$	$m_{15}(ABCD)$	$m_{14}(ABCD')$
10	$m_8(AB'C'D')$	$m_9(AB'C'D)$	$m_{11}(AB'CD)$	$m_{10}(AB'CD')$

四变量卡诺图的另一种形式

$f(A, B, C, D)$

AB CD	00	01	11	10
00	$m_0(A'B'C'D')$	$m_4(A'BC'D')$	$m_{12}(ABC'D')$	$m_8(AB'C'D')$
01	$m_1(A'B'C'D)$	$m_5(A'BC'D)$	$m_{13}(ABC'D)$	$m_9(AB'C'D)$
11	$m_3(A'B'CD)$	$m_7(A'BCD)$	$m_{15}(ABCD)$	$m_{11}(AB'CD)$
10	$m_2(A'B'CD')$	$m_6(A'BCD')$	$m_{14}(ABCD')$	$m_{10}(AB'CD')$

将函数表示成最小项相或的形式

$$Y = \bar{A}\bar{B}\bar{C}D + \bar{A}B\bar{D} + ACD + A\bar{B}$$

$$= \bar{A}\bar{B}\bar{C}D + \bar{A}B(C + \bar{C})\bar{D} + A(B + \bar{B})CD \\ + A\bar{B}(C + \bar{C})(D + \bar{D})$$

$$= \bar{A}\bar{B}\bar{C}D + \bar{A}BC\bar{D} + \bar{A}B\bar{C}\bar{D} + ABCD + A\bar{B}CD \\ + A\bar{B}CD + A\bar{B}C\bar{D} + A\bar{B}\bar{C}D + A\bar{B}\bar{C}\bar{D}$$

$$= m_1 + m_6 + m_4 + m_{15} + m_{11} \\ + m_{11} + m_{10} + m_9 + m_8$$

$$= \sum m_i (i = 1, 4, 6, 8, 9, 10, 11, 15)$$

用卡诺图表示逻辑函数

$$Y = \sum m_i (i = 1, 4, 6, 8, 9, 10, 11, 15)$$

CD \ AB	00	01	11	10
00		1		
01	1			1
11			1	
10	1	1	1	1

CD \ AB	00	01	11	10
00	0	1	0	0
01	1	0	0	1
11	0	0	1	0
10	1	1	1	1

本节课目录

- 数的编码与二进制表示
- 算术与函数的布尔代数表示
- 布尔表达式的化简

逻辑化简举例

$$Y = AC + \bar{B}C + B\bar{D} + \textcolor{red}{C\bar{D}} + A(B + \bar{C}) + \textcolor{red}{\bar{A}BC\bar{D}} + A\bar{B}DE$$

$$A + AB = A$$

$$= AC + \textcolor{red}{\bar{B}C} + B\bar{D} + C\bar{D} + \textcolor{red}{A\bar{B}\bar{C}} + A\bar{B}DE$$

$$A + \bar{A}B = A + B$$

$$= \textcolor{red}{AC} + \bar{B}C + B\bar{D} + C\bar{D} + \textcolor{red}{A} + \textcolor{red}{A\bar{B}DE}$$

$$A + AB = A$$

$$= A + \textcolor{red}{\bar{B}C} + \textcolor{red}{B\bar{D}} + \textcolor{red}{C\bar{D}}$$

$$AB + \bar{A}C + BC = AB + \bar{A}C$$

$$= A + \bar{B}C + B\bar{D}$$

Consensus theorem 一致性定理(多数定理)

化简动机

- 规范形式使得逻辑函数复杂化，其对应门电路也非常复杂

A	B	F
0	0	0
0	1	0
1	0	1
1	1	1

$$F = AB' + AB = A(B' + B) = A$$

$$F = A'B' + AB' = (A' + A)B' = B'$$

A	B	F
0	0	1
0	1	0
1	0	1
1	1	0

两级逻辑化简

有其他方法加速化简过程吗？

- 卡诺图
- （可选）CAD工具
 - Q-M算法

卡诺图化简法

- 相邻两个最小项仅有1个输入逻辑变量的取值不同，互补

BC	00	01	11	10
A				
0	0	0	1	1
1	0	0	1	1

Only B input is different
只有C这一输入逻辑
变量取值不同

只有A这一输入逻辑变量取值不同

卡诺图化简法

- **相邻**两个最小项仅有1个输入逻辑变量的取值不同，互补
- 如果这两个相邻最小项都使输出为1，则这两个最小项可合并，消除一个输入逻辑变量

BC	00	01	11	10
A				
0	0	0	1	1
1	0	0	1	1

$$ABC + ABC' = AB \rightarrow$$

可以合并为AB，与C的值无关

卡诺图化简法

- **相邻**两个最小项仅有1个输入逻辑变量的取值不同，互补
- 如果这两个相邻最小项都使输出为1，则这两个最小项可合并，消除一个输入逻辑变量
- 如果有四个相邻的最小项都使得输出函数取值为1，则这四个最小项可以合并，消除两个输入逻辑变量

BC	00	01	11	10
A				
0	0	0	1	1
1	0	0	1	1

$A'B$

可以合并为B，
与A、C值无关

AB

卡诺图化简法

- 如何保证卡诺图化简结果是最简的？ $\Rightarrow$ “蕴含” 的概念

BC	00	01	11	10
A				
0	0	0	1	1
1	0	0	1	1

A purple box highlights the four cells where A=1 and B=1 (the last two columns), illustrating the concept of "implication" (蕴含) in Karnaugh map simplification.

Implicant 蕴含项

- 如果 $F=1$ 则 $G=1$, 那么 F 是 G 的蕴含项 (Implicant) : $F \Rightarrow G$
 - 助记: 【 F 有效/成立】 “implies” 【 G 有效/成立】

Function F

BC	00	01	11	10
A				
0	0	0	0	1
1	0	0	0	1

$$F = BC' (=A'BC' + ABC')$$

Function G

BC	00	01	11	10
A				
0	0	0	0	1
1	0	0	1	1

$$G = BC' + ABC$$

$$F=1 \Rightarrow G=1$$

Prime Implicant 本原蕴含项

- 函数F的蕴含项P，如果不存在一个蕴含项 $Q \neq P$ ，使得 $P \Rightarrow Q \Rightarrow F$ ，则称P为本原蕴含项

BC	00	01	11	10
A				
0	0	0	0	1
1	0	0	1	1

①

②

③

②/③: 本原蕴含项

①: 蕴含项, 非本原蕴含项

Essential Prime Implicant本质本原蕴含项

- 如果一个函数中某个元素(最小项)仅由一个本原蕴含项覆盖, 则该本原蕴含项为本质本原蕴含项

$$Y = BC' + AB$$

②/③: 本原蕴含项

BC	00	01	11	10
A				
0	0	0	0	1
1	0	0	1	1

②/③: 本质本原蕴含项, 因为
 $A'BC'$ 和 ABC 分别只能由②和③覆盖

寻找本质本原蕴含项举例1

AB \ CD	00	01	11	10
00	0	1	1	0
01	1	1	1	0
11	1	0	1	1
10	0	0	1	1

$$Y = BC' + AC + A'B'D$$

不能被其它本原蕴含项覆盖

本原蕴含项: BC' , AC , AB , $A'C'D$, $A'B'D$, $B'CD$

谁是本质本原蕴含项?

本质本原蕴含项: BC' , AC

寻找本质本原蕴含项举例2

AB \ CD	00	01	11	10
00	0	0	1	0
01	1	1	1	0
11	0	1	1	1
10	0	1	0	0

$$Y = ACD + ABC' + A'C'D + A'BC$$

○ 只能被一个本原蕴含项覆盖

本原蕴含项: BD , ACD , ABC' , $A'C'D$, $A'BC$

谁是本质本原蕴含项?

非本质本原蕴含项: BD

寻找本质本原蕴含项举例3

$$Y = A\bar{C} + \bar{A}C + \bar{B}C + B\bar{C}$$

$$= A(B + \bar{B})\bar{C} + \bar{A}(B + \bar{B})C + (A + \bar{A})\bar{B}C + (A + \bar{A})B\bar{C}$$

$$= AB\bar{C} + A\bar{B}\bar{C} + \bar{A}BC + \bar{A}\bar{B}C + A\bar{B}C + \bar{A}BC + AB\bar{C} + \bar{A}B\bar{C}$$

$$= m_6 + m_4 + m_3 + m_1 + m_5 + m_1 + m_6 + m_2$$

$$= \sum m(1, 2, 3, 4, 5, 6)$$

谁是本质本原蕴含项？ 无！

BC	00	01	11	10
A				
0	0	1	1	1
1	1	1	0	1

本原蕴含项: $A'C$, $A'B$, AB' , AC' , $B'C$, BC'

寻找本质本原蕴含项举例3

$$Y = A\bar{C} + \bar{A}C + \bar{B}C + B\bar{C} = \sum m(1,2,3,4,5,6)$$

找到覆盖所有最小项的本质本原蕴含项

BC	00	01	11	10
A				
0	0	1	1	1
1	1	1	0	1

$$Y = A\bar{B} + \bar{A}C + B\bar{C}$$

BC	00	01	11	10
A				
0	0	1	1	1
1	1	1	0	1

$$Y = A\bar{C} + \bar{B}C + \bar{A}B$$

寻找本质本原蕴含项举例4

$$Y = ABC + ABD + A\bar{C}\bar{D} + \bar{C}\bar{D} + A\bar{B}C + \bar{A}C\bar{D}$$

		$\overbrace{\hspace{1.5cm}}^D$			
CD		00	01	11	10
AB					
00		1			1
01		1			1
11		1	1,1	1,1	1
10		1	1	1	1
$\underbrace{\hspace{1.5cm}}^A$				$\underbrace{\hspace{1.5cm}}^C$	
		$\left. \vphantom{\begin{matrix} 1 \\ 1 \\ 1, 1 \\ 1 \end{matrix}} \right\} B$			

寻找本质本原蕴含项举例4

$$Y = A + \bar{D}$$

CD \ AB	00	01	11	10
00	1	0	0	1
01	1	0	0	1
11	1	1	1	1
10	1	1	1	1

Diagram illustrating the Karnaugh map for the function $Y = A + \bar{D}$. The map shows the following groupings:

- A red box highlights the group of cells where $A = 1$ (columns 00 and 10), representing the term A .
- A purple box highlights the group of cells where $D = 0$ (rows 00 and 01), representing the term $\bar{D}$.

寻找本质本原蕴含项举例4

$$Y = A + \bar{D}$$

CD \ AB	00	01	11	10
00	1	0	0	1
01	1	0	0	1
11	1	1	1	1
10	1	1	1	1

A

$$\bar{Y} = \bar{A}D$$

$$Y = \bar{\bar{Y}} = \overline{\bar{A}D} = A + \bar{D}$$

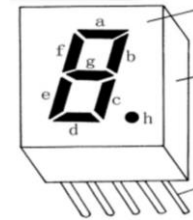
$\bar{D}$

用卡诺图化简举例

$$\begin{aligned} Y &= A\bar{C} + \bar{A}C + \bar{B}C + B\bar{C} \\ &= A(B + \bar{B})\bar{C} + \bar{A}(B + \bar{B})C + (A + \bar{A})\bar{B}C + (A + \bar{A})B\bar{C} \\ &= AB\bar{C} + A\bar{B}\bar{C} + \bar{A}BC + \bar{A}\bar{B}C + A\bar{B}C + \bar{A}B\bar{C} + AB\bar{C} + \bar{A}B\bar{C} \\ &= m_6 + m_4 + m_3 + m_1 + m_5 + m_1 + m_6 + m_2 \\ &= \sum m(1, 2, 3, 4, 5, 6) \end{aligned}$$

Don't care 无关项

- 一些例子
 - BCD累加电路
 - 十进制数字到七段显示译码电路
 - 输入取值组合1010~1111在实际电路中不出现
- **D**代表无关项表示的最大项
- **d**代表无关项表示的最小项



BCD累加电路的真值表

Function: $Y_8Y_4Y_2Y_1 = ABCD + 1$ (循环表示)

Input				Output				Input				Output			
A	B	C	D	Y_8	Y_4	Y_2	Y_1	A	B	C	D	Y_8	Y_4	Y_2	Y_1
0	0	0	0	0	0	0	1	1	0	0	0	1	0	0	1
0	0	0	1	0	0	1	0	1	0	0	1	0	0	0	0
0	0	1	0	0	0	1	1	1	0	1	0	x	x	x	x
0	0	1	1	0	1	0	0	1	0	1	1	x	x	x	x
0	1	0	0	0	1	0	1	1	1	0	0	x	x	x	x
0	1	0	1	0	1	1	0	1	1	0	1	x	x	x	x
0	1	1	0	0	1	1	1	1	1	1	0	x	x	x	x
0	1	1	1	1	0	0	0	1	1	1	1	x	x	x	x

BCD累加电路的布尔表达式

- $Y_1 = D'$
- 电路的任何实际实现输入都会产生一定的输出（包括无关项）

CD AB	00	01	11	10
00	000 1	0010	0100	001 1
01	010 1	0110	1000	011 1
11	xxx 1	xxxx	xxx x	xxx 1
10	100 1	0000	xxx x	xxx 1

无关项在逻辑化简时，可以任意假设其为0或1

用卡诺图得到最简或与表达式

CD \ AB	00	01	11	10
00	0	0	1	1
01	1	1	0	1
11	×	×	×	×
10	1	1	×	×

此例
与前面
的BCD
累加器
的例子
无关

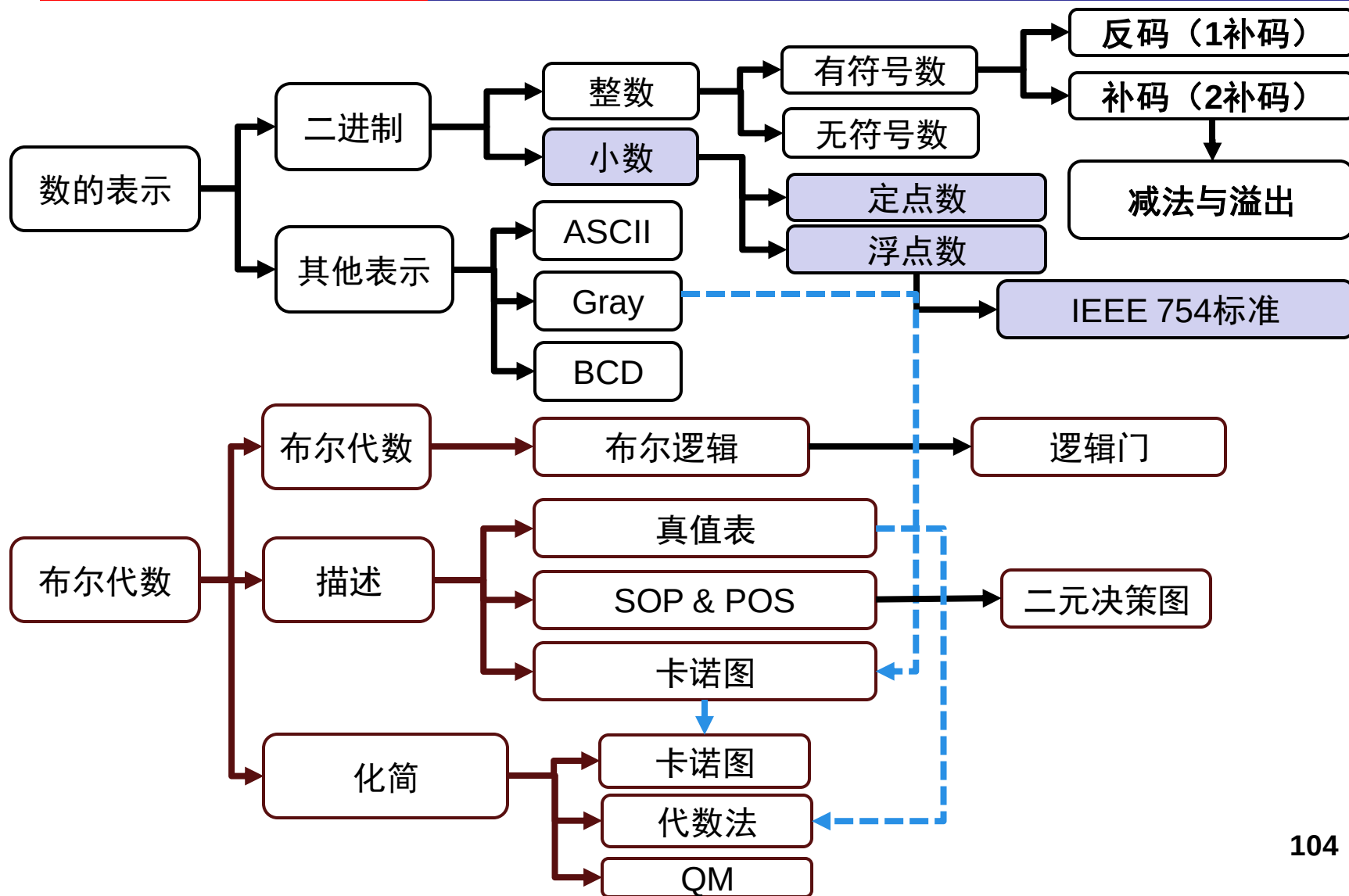
$$\bar{Y} = \bar{A}\bar{B}\bar{C} + BCD$$

$$Y = \bar{\bar{Y}} = (A + B + C) \cdot (\bar{B} + \bar{C} + \bar{D}) \quad \text{POS Form}$$

Quine-McCluskey法

- 由W.V. Quine和Edward J. McCluskey提出的表格形式的化简方法
- 可用计算机辅助设计
- 与卡诺图方法一致，找出相邻最小项，消除变量因子
- 用于多输入变量的函数化简

数的表示与布尔代数



总结

- **To know**
 - **Number system**
- **To design**
 - **Simplified Boolean logic and circuits**
- **To celebrate**
 - **The beauty of Boolean methodology**

作业 (1/2)

- 鼓励讨论问题，但请独立完成作业
- 提交到网络学堂
- 作业1
 - 请计算并补全以下表格

十进制	二进制	十六进制	8421 BCD码
124			
	0011 1011		
		0xB1	
			0011 1001

- 用8位二进制有符号整数表示十进制数 -34_{10} ， 59_{10} 的原码和2补码

作业 (2/2)

- 作业2

- 分别用代数化简法和卡诺图化简法化简逻辑函数：

$$F = A'B'C' + A'B'C + A'BC + ABC + A'BC'$$

- 已知函数： $f(A, B, C) = \sum m(0, 1, 2, 4, 5)$

- (1)将该函数转化为最大项表示的POS形式
 - (2)使用卡诺图将该函数化简为最简SOP形式
 - (3)使用下表提供的逻辑门，如果要以最小面积实现该函数的计算功能应如何设计电路，此时面积是多少？如果考虑最小延时呢？(注：门名称后的数字代表输入数；电路输入信号仅包括A,B,C)

逻辑门	延时(ps)	面积(μm^2)
NOT	18	8
AND2	50	25
NAND2	30	15
NAND4	70	30

逻辑门	延时(ps)	面积(μm^2)
OR2	55	26
NOR2	35	16
AND4	90	40
OR4	100	42
NOR4	80	32

自己练习：含无关项的最大/小项表示

- 已知逻辑函数 $f(W, X, Y, Z) = \prod M(0, 2, 4, 5, 9, 13, 14) \cdot D(7, 10)$ ，其中 M_i 是最大项（例， $M_3 = W + X + Y' + Z'$ ）， D_j 是无关项，将逻辑函数转换为最小项表达的SOP形式？

$$\begin{aligned} f(W, X, Y, Z) \\ = \sum m(1, 3, 6, 8, 11, 12, 15) + d(7, 10) \end{aligned}$$